

Section-Wise Code Explanation

1. login page.py: User Authentication Interface

This file manages the initial access point to the system, focusing purely on GUI setup and fixed-credential validation.

Section	Key Functionality	Details
Imports & Setup	GUI, Imagery, and Process Control	Imports tkinter for the user interface, PIL/Pillow for handling the background image (biometric_bg.jpg), and subprocess/sys/os to launch the next application module.
GUI Construction	Layout and Styling	Creates a two-panel layout: a static blue left panel for branding and a light right panel containing the login fields. The GUI is set to a fixed size (950x550).
Authentication Logic	Hardcoded Credentials	The login() method retrieves the entered username and password. It checks against the hardcoded credentials: "admin" and "admin" .
Application Flow	Module Launch	Upon successful authentication, the current window is destroyed (self.root.destroy()), and the open_homepage() method uses subprocess.Popen to launch the main menu script, homepage.py.

2. homepage.py: Main Menu Dashboard

This file serves as the central hub, presenting the user with options to access the two specialized disease detection modules.

Section	Key Functionality	Details
GUI Setup & Header	Branding and Navigation	Initializes the Tkinter window (1000x600) and displays a prominent header summarizing the "Cross-Modal Disease Prediction System."
Module Selection	Button Configuration	Creates two main buttons in a central frame: "Pupillometry-Based Genetic Disease Detection" and "Neurological Disease Prediction (Ataxia FusionNet)."
Button Handlers	Process Spawning	The <code>open_disease_detection()</code> and <code>open_neurology_module()</code> methods are responsible for launching their respective Python scripts (<code>dd.py</code> and <code>neurology.py</code>) using <code>subprocess.Popen</code> , allowing them to run independently.

3. disease detection.py: Pupillometry-Based Genetic Disease Detector

This is the main machine learning module, designed for training and testing various classifiers on pupillometric data.

Section	Key Functionality	Details
Imports & Compatibility	ML and Deep Learning Libraries	Imports comprehensive libraries including pandas, matplotlib, Scikit-learn (SVM, Ensembles, Split), and Keras (LSTM, BiLSTM). Includes compatibility fixes for scipy.linalg.pinv2 and sklearn_extensions.
Data Handling (upload, filtering)	Preprocessing and Feature Engineering	The filtering() function is critical: it iterates through files in a dataset directory, reads raw pupil size data, and extracts specific features (MAX, MIN, DELTA, CH, LATENCY, MCV) every 100 data points. It assigns a binary label (0 or 1) based on threshold checks (>500).
Data Preparation (featuresReduction)	Feature Scaling and Splitting	Loads the extracted features, normalizes them using normalize(), and performs an 80/20 train-test split for both left and right eye data.
Model Training & Comparison	Multiple Algorithms	Implements and trains six distinct models: Right SVM, Left SVM, Ensemble (Voting Classifier), Extreme Learning Machine (ELM), LSTM, and BiLSTM . Each function calculates and stores the accuracy score globally.
Prediction & Visualization	Results Output	The predict() function loads an external test file and uses the trained

		Ensemble Classifier to output predictions. The <code>graph()</code> function displays a Matplotlib bar chart comparing the accuracy of all six trained models.
--	--	--

4. neurology.py: Ataxia Detection Module

This module is specialized for neurological diagnosis using a specific PyTorch deep learning architecture (FusionNet) designed to process gait/kinematic data.

Section	Key Functionality	Details
Model Definition	PyTorch FusionNet Architecture	Defines the AtaxiaFusionNet , an <code>nn.Module</code> with two distinct branches: a BiLSTM branch for sequence/time-series data (gait patterns) and an MLP branch for static kinematic features. The outputs are concatenated in a final Fusion Layer for classification.
GUI and Setup	Prediction Interface	Sets up a Tkinter GUI with a large <code>ScrolledText</code> area for output. It defines an internal dictionary (<code>ataxia_info</code>) containing pre-written information about Cerebellar Ataxia.
Prediction Logic (predict_data)	Data Transformation and Inference	The core function for prediction: it loads the test CSV, uses <code>StandardScaler</code> for normalization, and performs padding and reshaping to convert the 2D feature vector into the

		3D tensor format required by the BiLSTM. It then loads the pre-trained model (ataxia_fusion_model.pth) and runs prediction inference.
Results Reporting	Detailed Output	Calculates total counts and confidence scores. For records predicted as 'ataxia,' it logs a detailed message, including the symptoms, cause, prevention, and treatment pulled from the ataxia_info dictionary . It saves the full predictions to outputs/final_predictions.csv.
Visualization	Graph Display	The show_graphs() function attempts to load and display pre-existing image files (.png) using Matplotlib's plt.imread for showing supplementary visualization results.